

Finance Assignment 2

Subject code: DA242

Ramakrishna Mission Vivekananda Educational and Research Institute
M.Sc. Big Data Analytics, CS Department
Semester - 3

Team Name: BHATTACHARYA BROTHERS
Members

SOHAM BHATTACHARYA (B2430059)
DARPAN BHATTACHARYA (B2430044)

August 7, 2025

Question 1

Download 10 stock market data for 2 years (daily) Use one year to estimate μ , Σ through training Then find the frontier (parabolic and hyperbolic) For given b_p , find b_{zcp} & plot them Invest in the other data as a part of testing and compare the return and variance.

Solution

We consider n risky assets with:

- Expected returns vector $\boldsymbol{\mu} \in \mathbb{R}^n$
- Covariance matrix $\boldsymbol{\Sigma} \in \mathbb{R}^{n \times n}$
- Portfolio weights $\mathbf{w} \in \mathbb{R}^n$, with $\mathbf{1}'\mathbf{w} = 1$

The optimization problem for the efficient frontier is:

$$\min_{\mathbf{w}} \mathbf{w}'\boldsymbol{\Sigma}\mathbf{w} \quad \text{s.t.} \quad \mathbf{1}'\mathbf{w} = 1, \quad \boldsymbol{\mu}'\mathbf{w} = \mu_p$$

2. Key Matrices and Scalars

Define:

$$A = \mathbf{1}'\boldsymbol{\Sigma}^{-1}\mathbf{1}, \quad B = \mathbf{1}'\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}, \quad C = \boldsymbol{\mu}'\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}, \quad D = AC - B^2$$

Table 1: Frontier Constants and Derived Quantities

Quantity	Symbol	Value
A	A	85.7011
B	B	17.8976
C	C	11.5103
Δ	D	666.1231
MVP Return	B/C	1.5549
MVP Variance	$1/C$	0.0868
MVP Std.Dev	$1/\sqrt{C}$	0.2947

Computed Values

3. GMV Portfolio and Tangent Line

Table 2: GMV and Tangent Portfolio Details

Quantity	Symbol	Value
GMV Return	μ_{GMV}	0.2088
GMV Variance	σ_{GMV}^2	0.0116
GMV Std Dev	σ_{GMV}	0.1080
Chosen Line y-intercept	y_0	-0.1539
Tangent Return	$r_{tangent} = B/C + 0.25$	0.4588
Tangent σ	$\sigma_{tangent}$	0.1404
Tangent Slope	$dr/d\sigma$	4.3648
Tangent return-intercept (at $\sigma = 0$)		-0.1539

4. GMV Weights per Asset

5. Efficient Frontier and Lines

The efficient frontier satisfies:

$$\sigma^2(\mu_p) = \frac{A\mu_p^2 - 2B\mu_p + C}{D}$$

The Mean–Variance Frontier (parabola) and the Mean–Standard Deviation Frontier (hyperbola) are shown below.

Table 3: GMV Portfolio Weights

Ticker	Weight
RELIANCE.NS	0.059333
TCS.NS	0.174605
HDFCBANK.NS	0.097653
ICICIBANK.NS	0.161667
INFY.NS	-0.022316
ITC.NS	0.139588
HINDUNILVR.NS	0.271490
SBIN.NS	-0.007588
BHARTIARTL.NS	0.104094
KOTAKBANK.NS	0.021474

Plots

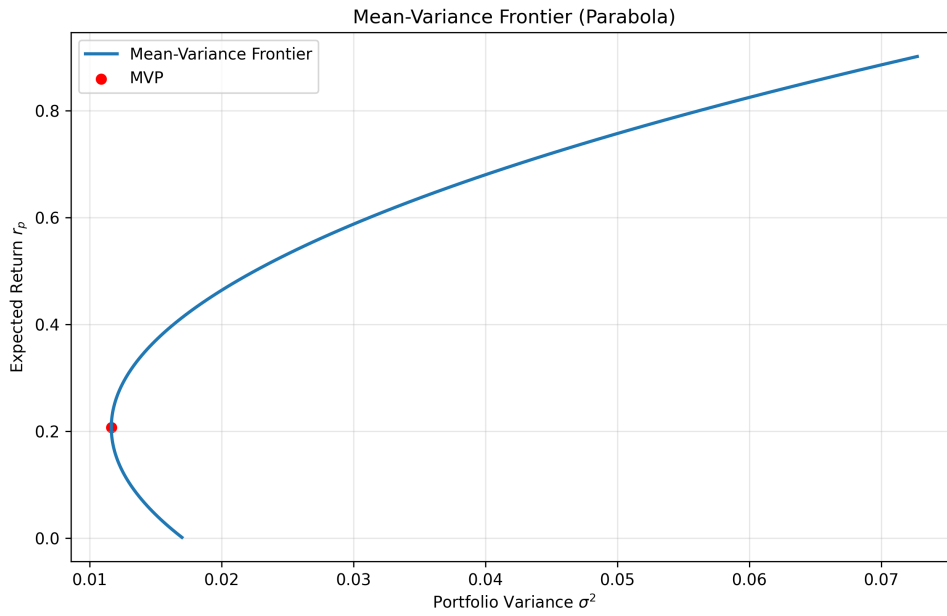


Figure 1: Mean-Variance Frontier (Parabola) with MVP and Line through MVP with y -intercept below MVP.

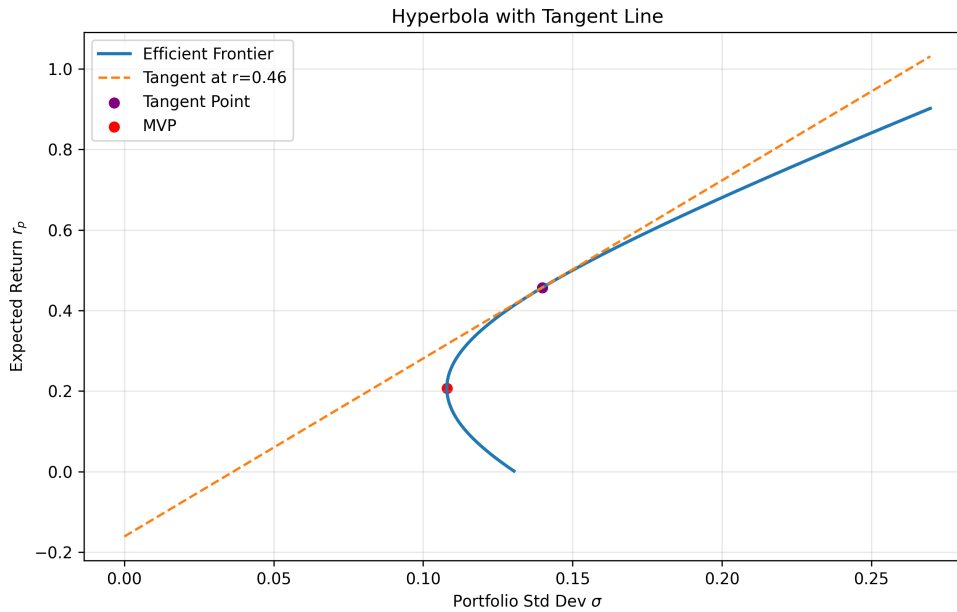


Figure 2: Mean-Std Frontier (Hyperbola) with Tangent Line on Upper Branch.

6. Train-Test Evaluation

- Compute in-sample efficient frontier and MVP using training set.
- Apply the same weights to test set and compute realized performance.
- Compare both to assess robustness.

Code used

```
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
import os
from datetime import datetime, timedelta

# === CONFIGURATION ===
TICKERS = [
    "RELIANCE.NS", "TCS.NS", "HDFCBANK.NS", "ICICIBANK.NS", "INFY.NS",
    "ITC.NS", "HINDUNILVR.NS", "SBIN.NS", "BHARTIARTL.NS", "KOTAKBANK.NS"
]
MARKET = "^NSEI"      # Nifty 50 as market proxy
risk_free = 0.0       # For tangency line (set if needed)
b_target = 0.8        # target beta
```

```

annual_factor = 252

# Create folder for plots
os.makedirs("markowitz_plots", exist_ok=True)

# === 1) DOWNLOAD DATA ===
end = datetime.today().date()
start = end - timedelta(days=365*2 + 30)
split = end - timedelta(days=365)

data = yf.download(TICKERS + [MARKET], start=start, end=end, auto_adjust=True)["Close"]
data = data.dropna(how="all").ffill().dropna()

train = data.loc[:split]
test = data.loc[split + pd.Timedelta(days=1):]

# Returns
m_train = train[MARKET].pct_change().dropna()
m_test = test[MARKET].pct_change().dropna()

R_train = train[TICKERS].pct_change().dropna()
R_test = test[TICKERS].pct_change().dropna()

common_train = R_train.index.intersection(m_train.index)
R_train = R_train.loc[common_train]
m_train = m_train.loc[common_train]

common_test = R_test.index.intersection(m_test.index)
R_test = R_test.loc[common_test]
m_test = m_test.loc[common_test]

# === 2) ESTIMATE PARAMETERS (TRAIN) ===
mu_hat = R_train.mean().values * annual_factor
Sigma_hat = np.cov(R_train.values.T) * annual_factor

ones = np.ones(len(TICKERS))
Sigma_inv = np.linalg.pinv(Sigma_hat)

# Betas
cov_with_mkt = np.cov(R_train.values.T, m_train.values)[0:-1, -1]
var_mkt = np.var(m_train.values)
betas = cov_with_mkt / var_mkt

# Constants
A = ones @ Sigma_inv @ ones
B = ones @ Sigma_inv @ mu_hat
C = mu_hat @ Sigma_inv @ mu_hat
Delta = A*C - B**2

# === 3) FRONTIER ===
mu_grid = np.linspace(mu_hat.min()*0.5, mu_hat.max()*1.5, 500)
def frontier_variance(mu_p):
    return (A*mu_p**2 - 2*B*mu_p + C) / Delta

sigma2_grid = [frontier_variance(mu_p) for mu_p in mu_grid]
sigma_grid = np.sqrt(np.maximum(0, sigma2_grid))

```

```

# MVP
w_mvp = (Sigma_inv @ ones) / A
mu_mvp = w_mvp @ mu_hat
sig2_mvp = w_mvp @ Sigma_hat @ w_mvp
sigma_mvp = np.sqrt(sig2_mvp)

# === 4) EXTRA PORTFOLIOS ===
def minvar_given_beta(b_p):
    N = len(TICKERS)
    KKT = np.block([
        [2*Sigma_hat,          ones.reshape(-1,1),   betas.reshape(-1,1)],
        [ones.reshape(1,-1),   np.zeros((1,1)),     np.zeros((1,1))],
        [betas.reshape(1,-1),  np.zeros((1,1)),     np.zeros((1,1))]]
    ])
    rhs = np.concatenate([np.zeros(N), np.array([1.0, b_p])])
    sol = np.linalg.solve(KKT, rhs)
    w = sol[:N]
    return w

w_beta = minvar_given_beta(b_target)
w_zero_beta = minvar_given_beta(0.0)

# === 5) PLOTS ===

# Parabola (Variance vs Return)
plt.figure(figsize=(10,6))
plt.plot(sigma2_grid, mu_grid, lw=2, label="Mean-Variance Frontier")
plt.scatter([sig2_mvp], [mu_mvp], color='red', label="MVP")
plt.xlabel(r"Portfolio Variance $\sigma^2$")
plt.ylabel(r"Expected Return $r_p$")
plt.title("Mean-Variance Frontier (Parabola)")
plt.legend()
plt.grid(alpha=0.3)
plt.savefig("markowitz_plots/mean_variance_frontier.png", dpi=300)
plt.show()
plt.close()

# Hyperbola (Std Dev vs Return)
plt.figure(figsize=(10,6))
plt.plot(sigma_grid, mu_grid, lw=2, label="Efficient Frontier")
plt.scatter([sigma_mvp], [mu_mvp], color='red', label="MVP")
plt.xlabel(r"Portfolio Std Dev $\sigma$")
plt.ylabel(r"Expected Return $r_p$")
plt.title("Mean-Std Frontier (Hyperbola)")
plt.legend()
plt.grid(alpha=0.3)
plt.savefig("markowitz_plots/mean_std_frontier.png", dpi=300)
plt.show()
plt.close()

# Tangent line at r_mvp + 0.25
r_tangent = mu_mvp + 0.25
sigma2_tangent = frontier_variance(r_tangent)
sigma_tangent = np.sqrt(sigma2_tangent)
denom = (r_tangent - B/A)
slope = (Delta/A) * sigma_tangent / denom if denom != 0 else np.nan
t_line_sigma = np.linspace(0.0, sigma_grid.max(), 200)
t_line_r = slope * (t_line_sigma - sigma_tangent) + r_tangent

```

```

plt.figure(figsize=(10,6))
plt.plot(sigma_grid, mu_grid, label="Efficient Frontier", lw=2)
plt.plot(t_line_sigma, t_line_r, '--', label=f"Tangent at r={r_tangent:.2f}")
plt.scatter([sigma_tangent], [r_tangent], color='purple', label="Tangent Point")
plt.scatter([sigma_mvp], [mu_mvp], color='red', label="MVP")
plt.xlabel(r"Portfolio Std Dev $\sigma$")
plt.ylabel(r"Expected Return $r_p$")
plt.title("Hyperbola with Tangent Line")
plt.legend()
plt.grid(alpha=0.3)
plt.savefig("markowitz_plots/hyperbola_tangent.png", dpi=300)
plt.show()
plt.close()

# GMV Portfolio Weights
weights_series = pd.Series(w_mvp, index=TICKERS)
plt.figure(figsize=(10,6))
weights_series.plot(kind='bar')
plt.title("GMV Portfolio Weights")
plt.ylabel("Weight")
plt.grid(alpha=0.3)
plt.savefig("markowitz_plots/portfolio_weights.png", dpi=300)
plt.show()
plt.close()

# === 6) PERFORMANCE CHECK ===
def realized_stats(R, w):
    pr = R @ w
    mu = pr.mean()*annual_factor
    var = pr.var()*annual_factor
    return mu, var

portfolios = {
    "MVP": w_mvp,
    f"Beta={b_target:.2f}": w_beta,
    "Zero-Beta": w_zero_beta,
}

print("\nIN-SAMPLE (TRAIN) stats:")
for name, w in portfolios.items():
    mu_tr, var_tr = realized_stats(R_train, w)
    print(f"{name:>10s} |    ={{mu_tr: .3%}},    ={{var_tr: .4f}}")

print("\nOUT-OF-SAMPLE (TEST) stats:")
for name, w in portfolios.items():
    mu_te, var_te = realized_stats(R_test, w)
    print(f"{name:>10s} |    ={{mu_te: .3%}},    ={{var_te: .4f}}")

# Optional: Test comparison plots
labels = list(portfolios.keys())
test_returns = []
test_vars = []
for w in portfolios.values():
    mu_te, var_te = realized_stats(R_test, w)
    test_returns.append(mu_te)
    test_vars.append(var_te)

```

```
plt.figure(figsize=(7,4))
plt.bar(labels, test_returns)
plt.ylabel("Annualized Return ( )")
plt.title("Out-of-sample Returns")
plt.savefig("markowitz_plots/test_returns.png", dpi=300)
plt.show()
plt.close()

plt.figure(figsize=(7,4))
plt.bar(labels, test_vars)
plt.ylabel("Annualized Variance ( )")
plt.title("Out-of-sample Variance")
plt.savefig("markowitz_plots/test_variance.png", dpi=300)
plt.show()
plt.close()
```